

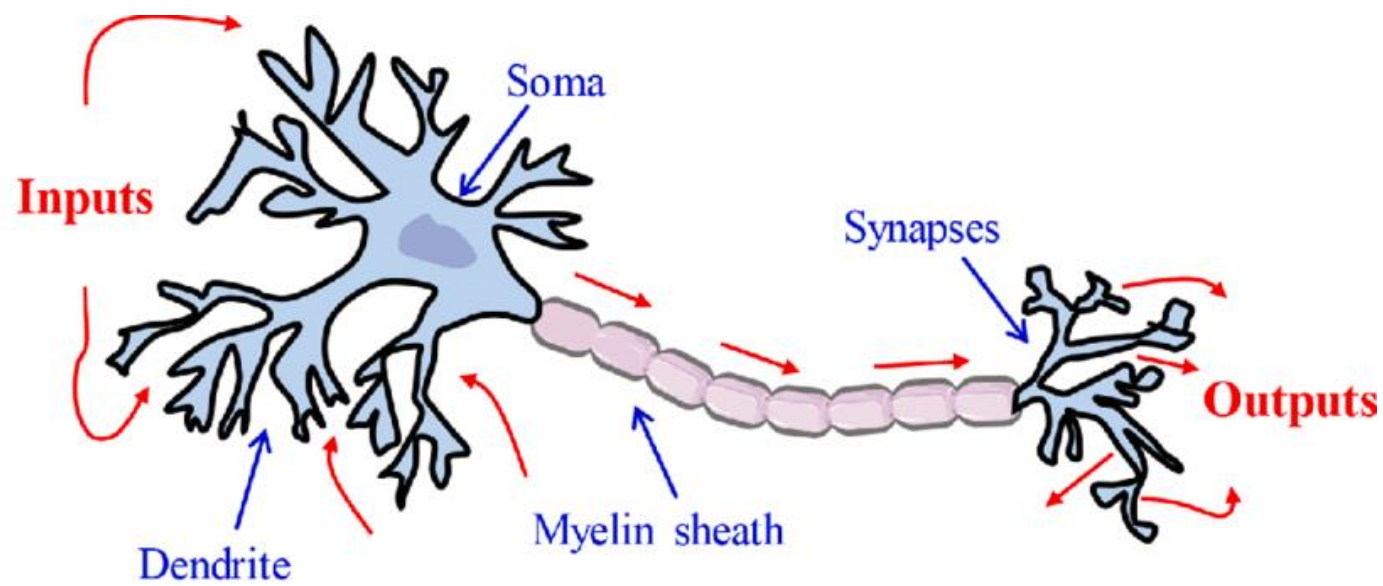
Artificial Neural Network

Natural Language Processing

Ayesha Enayet

Definition

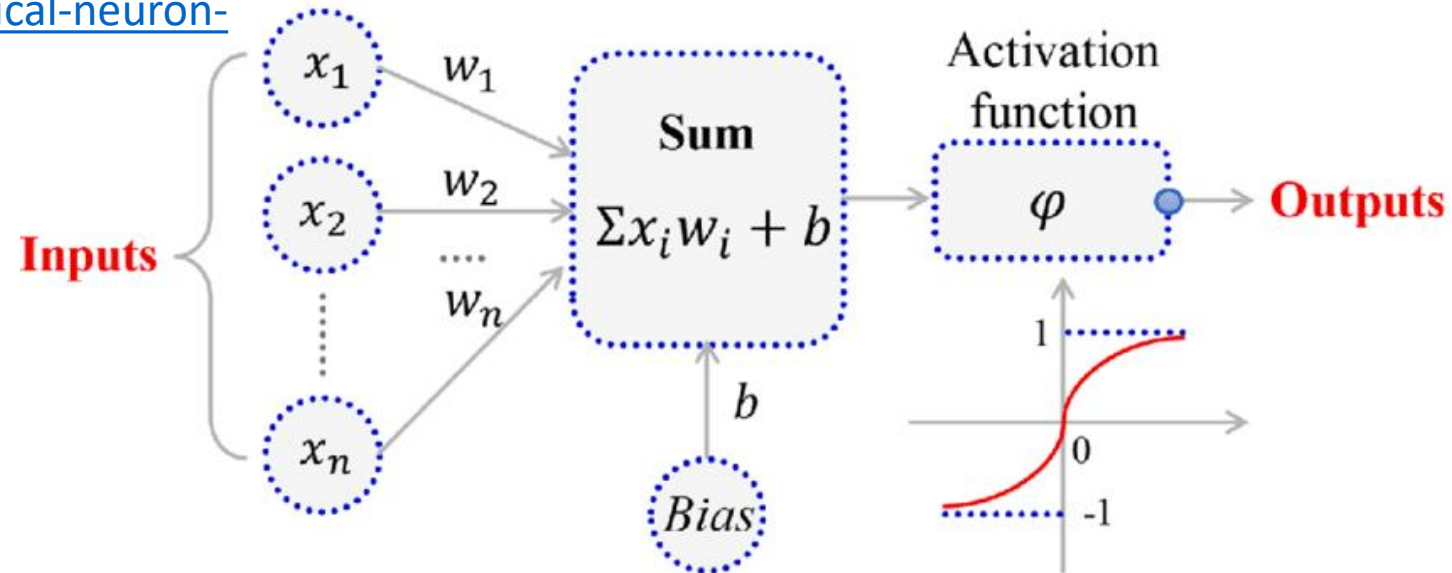
- A neural network is a method in artificial intelligence that teaches computers to process data in a way that is inspired by the human brain. It is a type of [machine learning](#) process that uses interconnected nodes or neurons in a layered structure that resembles the human brain. It creates an adaptive system that computers use to learn from their mistakes and improve continuously.
- <https://aws.amazon.com/what-is/neural-network/>



(a) Biological neuron

Taken from

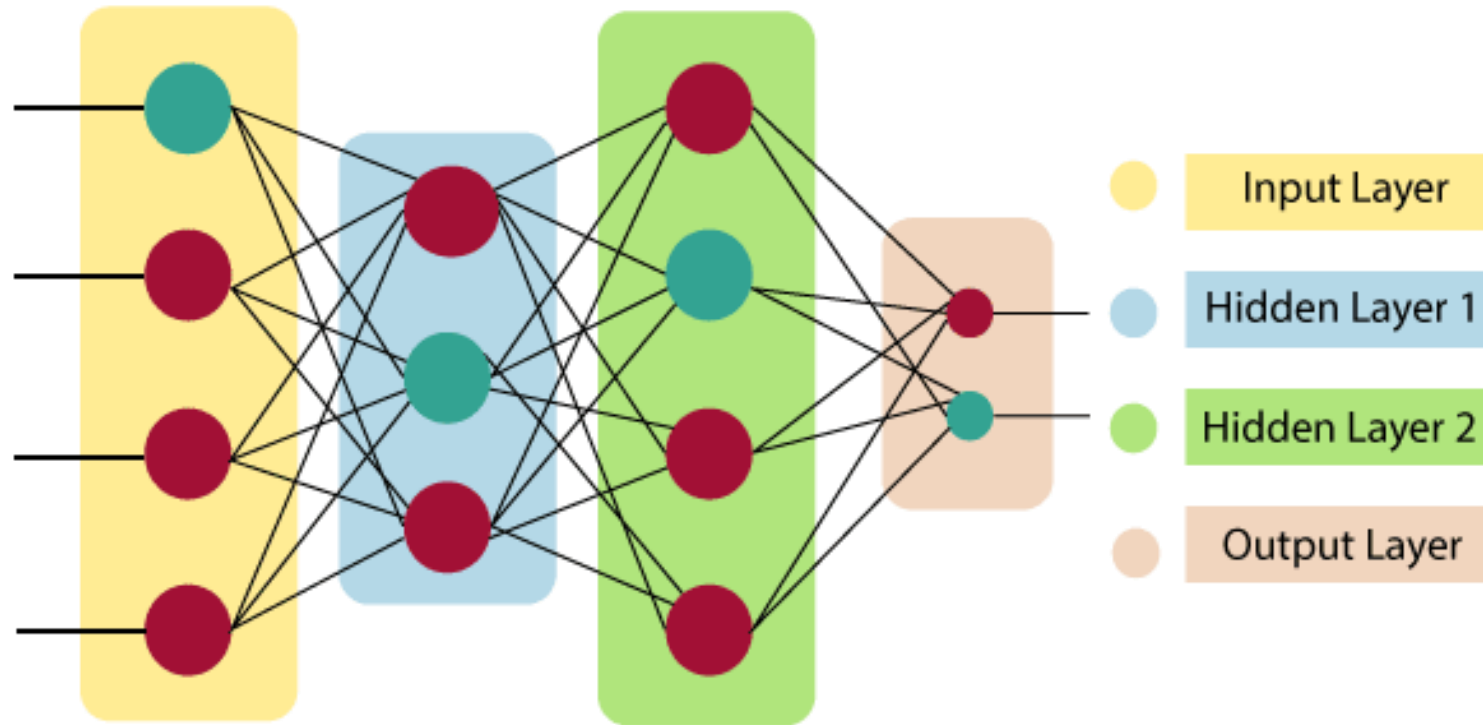
https://www.researchgate.net/figure/Comparison-between-biological-neuron-and-artificial-neuron-40_fig4_351372032



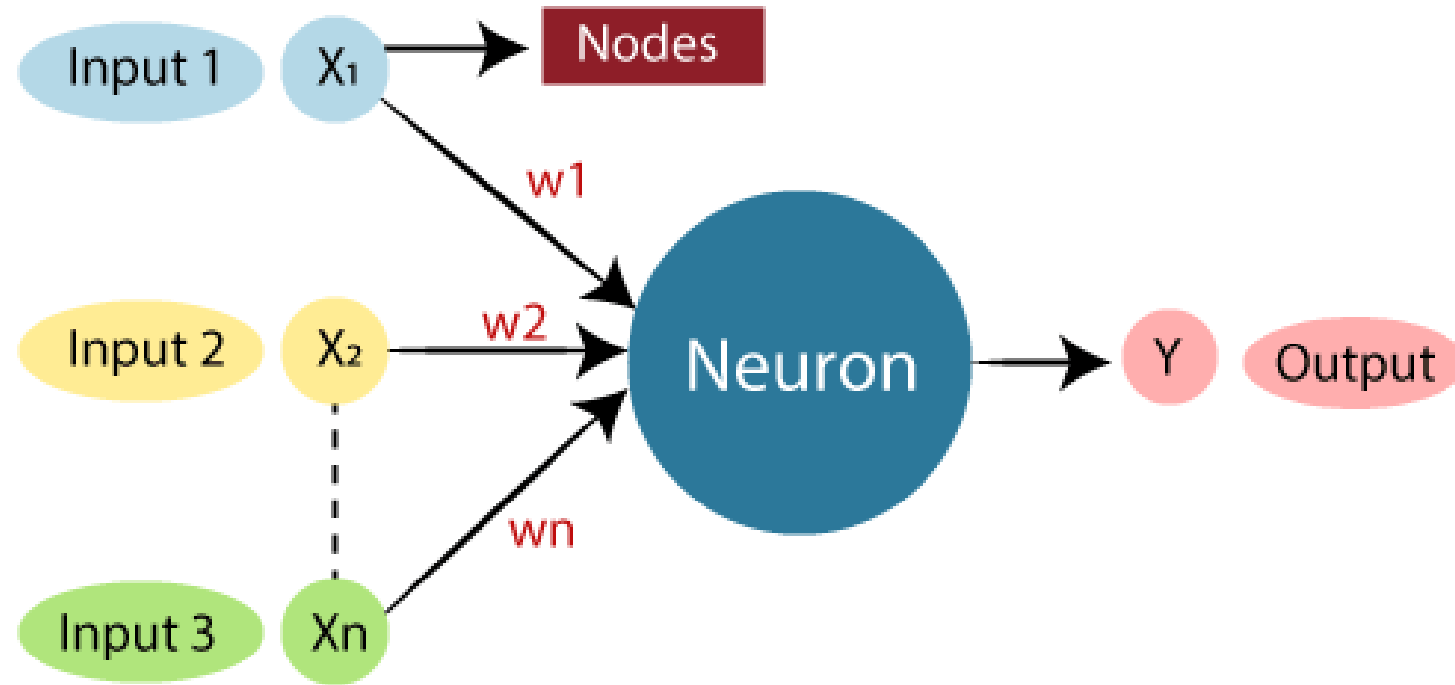
(b) Artificial neuron

Architecture

- Figure is taken from <https://www.javatpoint.com/artificial-neural-network>



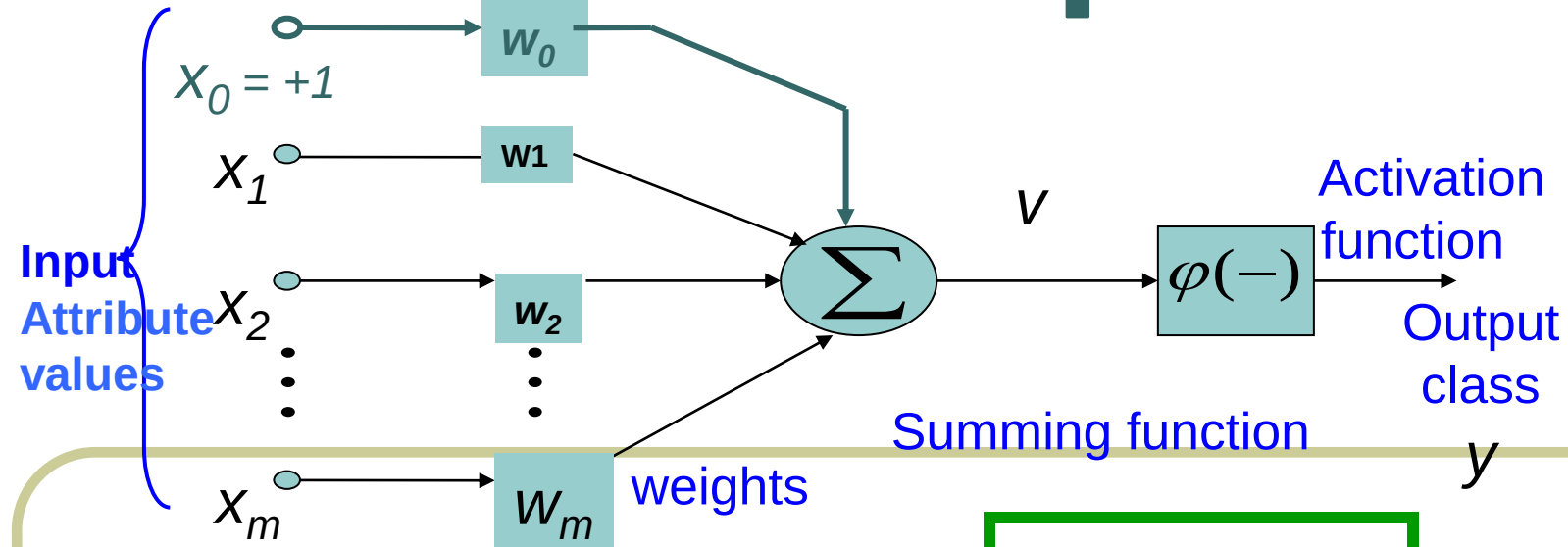
One neuron as a network



Working of one neuron

Credit : Lecture slides from **State University of New York at Stony Brook**

Bias as extra input



$$v = \sum_{j=0}^m w_j x_j$$
$$w_0 = b$$

Neuron with Activation

- The neuron is the basic information processing unit of a NN. It consists of:

- 1 A set of **links**, describing the neuron inputs, with **weights** $W_1, W_2, \dots, W_m$
2. An **adder** function (linear combiner) for computing the weighted sum of the inputs (real numbers):

$$u = \sum_{j=1}^m w_j x_j$$

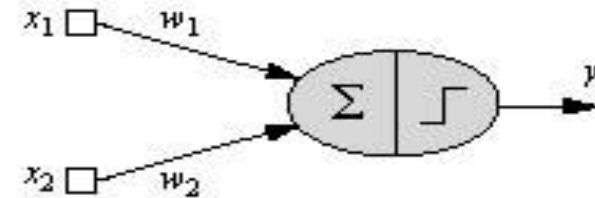


Fig1: an artificial neuron

- 3 **Activation function** : for limiting the amplitude of the neuron output.

$$y = \varphi(u + b)$$

Example

x1	x2	w1	w2	b	X (activation)
0.5	0.3	0.5	0.5	0.1	0.622

Weighted Sum = $0.5 * 0.5 + 0.3 * 0.5 + 0.1 = 0.5$

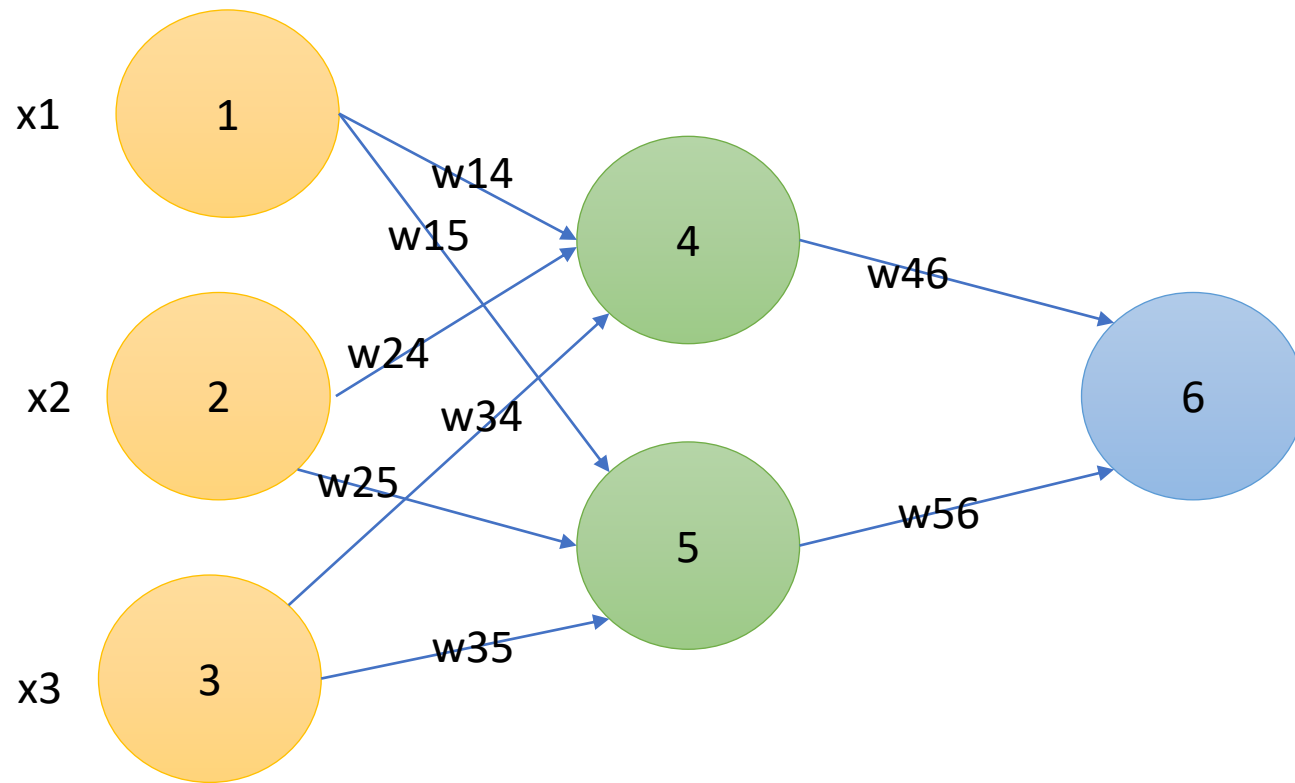
Activation function = $\frac{1}{1 + e^{-x}}$ (sigmoid function)

$Y = \Phi(X)$

$\Phi(X) = 1$ {where $X < 0.5$ }

$\Phi(X) = 0$ {where $X \geq 0.5$ }

Multi-layer neural network



Random Initialization of weights between -1 to 1
 Random Initialization of bias between -1 and 1

Activation function= $\frac{1}{1+e^{-x}}$
 (sigmoid function)

Input			H1						O		ϕ
x1	x2	x3	w14	w15	w24	W25	w34	w35	w46	w56	
0.5	0.2	0.6	-0.2	0.3	0.4	0.4	-0.5	0.4	0.6	0.2	

b4	b5	b6
-0.5	0.4	0.2

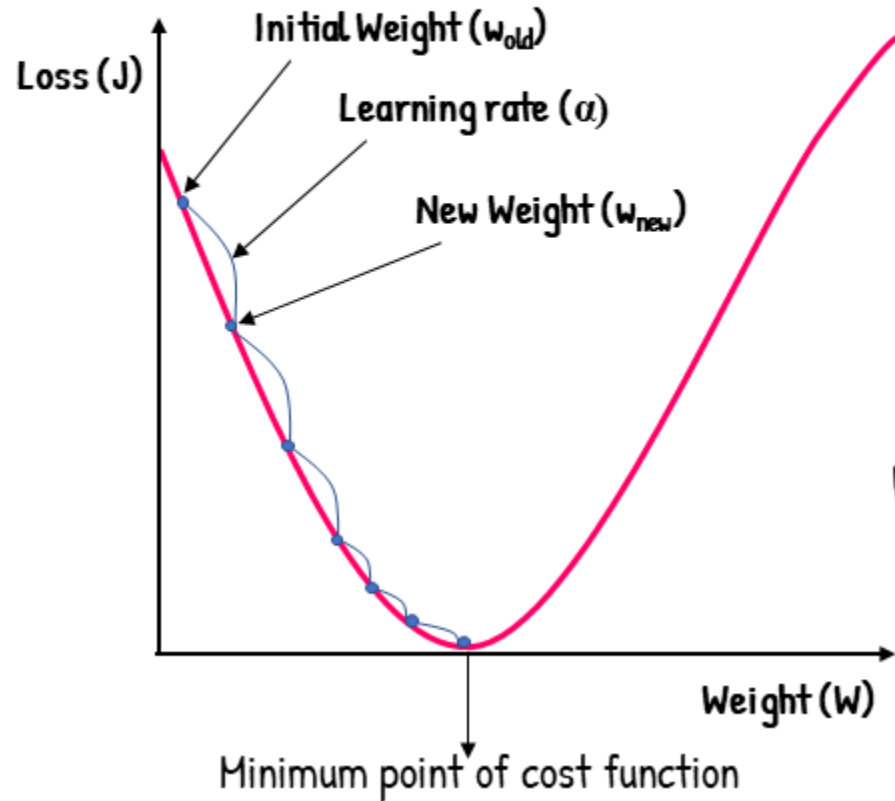
Neuron	Output O
4 $0.5 \cdot -0.2 + 0.2 \cdot 0.4 + 0.6 \cdot -0.5 + (-0.5) = -0.1 + 0.08 - 0.3 - 0.5 = -0.82$	0.305
5 $0.5 \cdot 0.3 + 0.2 \cdot 0.4 + 0.6 \cdot 0.4 + (0.4) = .15 + 0.08 + 0.24 + 0.4 = 0.87$	0.705
6 $0.305 \cdot 0.6 + 0.705 \cdot 0.2 + 0.2 = 0.524$	0.628

Input			H1						O		ϕ
x1	x2	x3	w14	w15	w24	W25	w34	w35	w46	w56	
0.5	0.2	0.6	-0.2	0.3	0.4	0.4	-0.5	0.4	0.6	0.2	

b4	b5	b6
-0.5	0.4	0.2

Backpropagation

Gradient Descent



$$w_{new} = w_{old} - \alpha \frac{\delta J}{\delta w}$$

Taken from <https://www.linkedin.com/pulse/understanding-gradient-descent-algorithm-its-role-linear-mhango-kjbvf/>

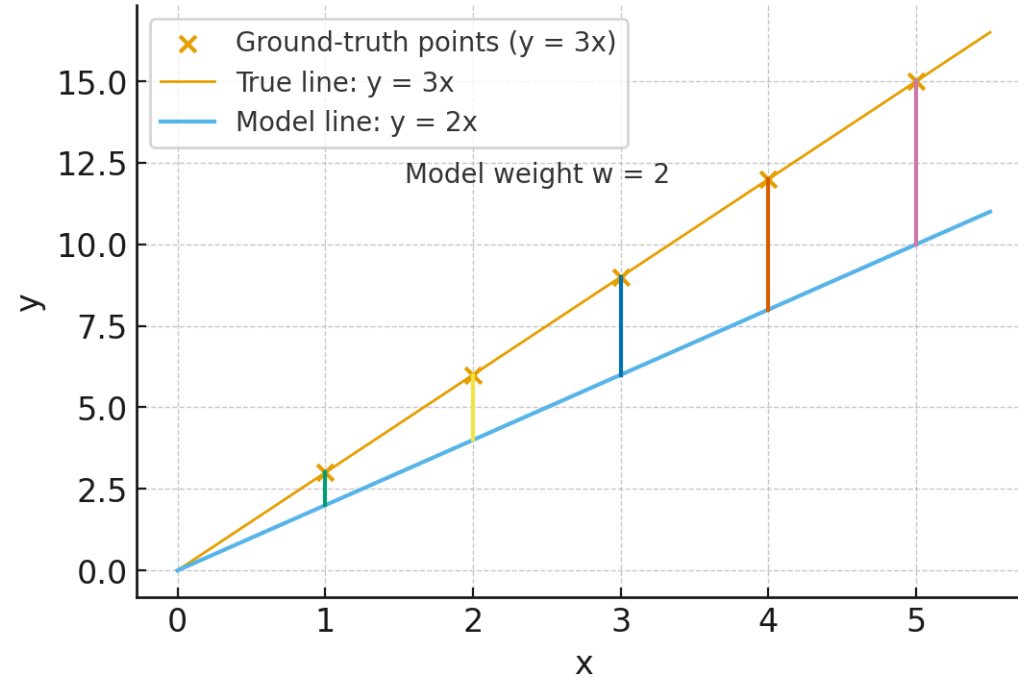
$$w_{\text{new}} = w_{\text{old}} - \alpha \frac{\delta J}{\delta w}$$

$\frac{\partial L}{\partial w_{ij}}$ is the **slope** (rate of change) of the loss L if you nudge just that one weight w_{ij} while holding all others fixed.

- **Sign:**
 - > 0 : increasing w_{ij} increases loss $\Rightarrow$ to reduce loss, **decrease** w_{ij} .
 - < 0 : increasing w_{ij} decreases loss $\Rightarrow$ to reduce loss, **increase** w_{ij} .
- **Magnitude:** how sensitive the loss is to that weight; bigger magnitude $\Rightarrow$ bigger effect.

Simplified Example to
explain derivative of loss
w.r.t weight

Ground Truth $y=3x$ vs Model $y=2x$ (Model weight $w = 2$)



Tiny example: one-neuron linear model $\hat{y} = wx$ with squared loss $L = \frac{1}{2}(\hat{y} - y)^2$.

$$\frac{dL}{dw} = (wx - y) x.$$

i.e. Error * input

If $w x > y$ (you overpredict), the slope is positive $\Rightarrow$ step $w \leftarrow w - \eta(wx - y)x$
decreases w and lowers the loss.

Updating weights

$$w_{\text{new}} = w_{\text{old}} - \alpha \frac{\delta J}{\delta w}$$

$$w_{ij} = w_{ij} + \Delta w_{ij}$$

$$\Delta w_{ij} = (l) \text{Err}_j O_i$$

Error

$$L \xrightarrow{\partial L / \partial O} O \xrightarrow{\partial O / \partial z} z.$$

Where z is pre-activation output and O is activation

So by the chain rule,

$$\frac{\partial L}{\partial z} = \frac{\partial L}{\partial O} \cdot \frac{\partial O}{\partial z}.$$

Error

- Loss (SSE/MSE form for one output unit): $L = \frac{1}{2} (T - O)^2$

1) Derivative w.r.t. the output O

$$\frac{\partial L}{\partial O} = \frac{1}{2} \cdot 2 (T - O) (-1) = O - T.$$

2) Chain rule to the pre-activation z

$$\frac{\partial L}{\partial z} = \frac{\partial L}{\partial O} \cdot \frac{\partial O}{\partial z}.$$

For sigmoid, $\frac{\partial O}{\partial z} = \sigma(z)(1 - \sigma(z)) = O(1 - O).$

Here's the full, step-by-step calculus derivation.

Let the sigmoid (logistic) function be

$$\sigma(z) = \frac{1}{1 + e^{-z}} = (1 + e^{-z})^{-1}.$$

Differentiate w.r.t. z using the chain rule:

1. Rewrite and differentiate:

$$\frac{d}{dz}\sigma(z) = \frac{d}{dz}(1 + e^{-z})^{-1} = -(1 + e^{-z})^{-2} \cdot \frac{d}{dz}(1 + e^{-z}) = -(1 + e^{-z})^{-2} \cdot (-e^{-z}) = \frac{e^{-z}}{(1 + e^{-z})^2}.$$

2. Express this purely in terms of $\sigma(z)$.

From $\sigma(z) = \frac{1}{1 + e^{-z}}$ we have

$$1 - \sigma(z) = 1 - \frac{1}{1 + e^{-z}} = \frac{e^{-z}}{1 + e^{-z}}.$$

Therefore,

$$\sigma(z)(1 - \sigma(z)) = \frac{1}{1 + e^{-z}} \cdot \frac{e^{-z}}{1 + e^{-z}} = \frac{e^{-z}}{(1 + e^{-z})^2}.$$

Comparing with step (1),

$$\sigma'(z) = \frac{d\sigma}{dz} = \sigma(z)(1 - \sigma(z))$$

Error

Output Neuron

$$Err_k = O_k (1 - O_k) (T_k - O_k)$$

T_k is our ground truth value in example, O_k is output that we get after applying the activation function

For the example that is under consideration $k=\{6\}$

Hidden layer Neuron

$$Err_j = O_j (1 - O_j) \sum_k Err_k w_{jk}$$

For the example that is under consideration $j=\{4,5\}$

Some concepts

$$Err_k = O_k(1 - O_k)(T_k - O_k)$$

What it is

- O_k : the neuron's output (after sigmoid).
- T_k : the target label.
- $O_k(1 - O_k)$: the derivative of the sigmoid, i.e., how sensitive the output is to its input.
- $(T_k - O_k)$: the raw prediction error (direction and size).

Multiply them and you get the derivative of the loss w.r.t. the neuron's **net input** z_k :

$\frac{\partial E}{\partial z_k} = -Err_k$ (sign conventions vary—many define $\delta_k = (T_k - O_k)O_k(1 - O_k)$ and then use it directly).

Why measure sensitivity?

- So we include the sensitivity not to “measure error,” but to scale the error by **how effective** a parameter change would be at reducing that error—the essence of gradient descent.

$$Err_k = O_k(1 - O_k)(T_k - O_k)$$

$$O_k(1 - O_k) = \sigma(z)(1 - \sigma(z)) = \frac{d\sigma(z)}{dz}$$

Interpretation:

- $(T - O)$ is the **raw error** (how wrong the prediction is).
- $O(1 - O)$ is the **sensitivity** (how much O would change if we nudged z).

Multiplying them gives the **right-sized learning signal**: big when we're wrong *and* the neuron is responsive; small when we're near correct or the sigmoid is saturated.

Error

$$Err_k = O_k (1 - O_k) (T_k - O_k)$$

$$Err_j = O_j (1 - O_j) \sum_k Err_k w_{jk}$$

	O	Error
4	0.305	=0.305(1-0.305)(0.0869*0.6)=0.011
5	0.705	=0.705(1-0.705)(0.0869*0.2)=0.705*.295*0.0174=0.0036
6	0.628	=0.628(1-0.628)(1-.628)=0.0869

Note: T_k is the ground truth, take $T_k=1$ for positive class

Updating weights

$$w_{\text{new}} = w_{\text{old}} - \alpha \frac{\delta J}{\delta w}$$

$$\Delta w_{ij} = (l) \text{Err}_j O_i$$

$$w_{ij} = w_{ij} + \Delta w_{ij}$$

$$\Delta w_{ij} = (l)Err_j O_i$$

$$w_{ij} = w_{ij} + \Delta w_{ij}$$

	O	Error	
4	0.305	0.011	W14=-0.2+(0.1*0.011*0.5) W24=0.4+ W34=-0.5+
5	0.705	0.0036	W15=0.3+ W25=0.4+ W35=0.4+
6	0.628	0.0869	W46= 0.6+(0.1*0.0869*0.305) W56=0.2+(0.1*0.0869*0.705)

Updating bias

$$\Delta \theta_j = (l) Err_j$$

$$\theta_j = \theta_j + \Delta \theta_j$$

Err(j)	b _j
4: 0.011	= -0.5 + (0.1 * 0.011)
5: 0.0036	= 0.4 + (0.1 * 0.0036)
6: 0.0869	= 0.2 + (0.1 * 0.0869)

Coding help

- https://colab.research.google.com/drive/1CONh3rp2KYWexKimYbJLcU2Y_bSCS_4s?usp=sharing
- Note: For the sake of understanding and assignment we can generate vector for each sentence by adding the one-hot encode vectors of all of its words. In case of repetition, count it once.

Updating the weight and neuron sensitivity

Because gradient descent doesn't update weights based on **error alone**—it updates them based on **how much a tiny change in a weight would change the loss**. That “how much” is exactly the neuron's **sensitivity** (the derivative).

Mathematically, for a weight w_{jk} into output unit k :

$$\frac{\partial E}{\partial w_{jk}} = \underbrace{\frac{\partial E}{\partial O_k}}_{\text{prediction error } (O_k - T_k)} \times \underbrace{\frac{\partial O_k}{\partial z_k}}_{\text{sensitivity } = O_k(1 - O_k)} \times \underbrace{\frac{\partial z_k}{\partial w_{jk}}}_{\text{input } o_j}.$$

Why the sensitivity term matters:

- **Correct credit assignment.**

Two neurons can have the same error but different responsiveness. If a neuron is *saturated* (sigmoid near 0 or 1), changing its weight barely changes the output, so the update should be **small**. If it's in the steep region (near 0.5), the same error should cause a **larger** update. The derivative encodes this.

- **Move the right amount (chain rule).**

Gradient descent must follow the true slope of the loss surface. The derivative $\frac{\partial O_k}{\partial z_k} \frac{\partial z_k}{\partial O_k}$ is part of that slope; dropping it would over/under-shoot and can prevent convergence.

- **Stability and efficiency.**

- Big slope $\Rightarrow$ larger step where the model is plastic (learns fast).
- Tiny slope $\Rightarrow$ cautious step in saturated regions (avoids wild, useless updates).

- **Backprop through layers.**

In hidden layers, sensitivity terms gate how much error signal flows backward. Units that could have influenced the output more (high slope) receive more “blame” and get bigger updates.